

深圳大学实验报告

课程名称： 算法设计与分析

实验项目名称： 图论——最大流应用问题

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 杨烜

报告人： 姚泽棉 学号： 2024150026

实验时间： 2026年06月12日—2026年07月03日

实验报告提交时间： 2026年06月28日

教务部制

一、实验目的

- (1) 掌握最大流算法思想。
- (2) 学会用最大流算法求解应用问题。

二、内容



1996 年 9 月 10 日,《旧金山纪事报》的体育版上登载了《巨人队正式告别 NL 西区比赛》一文,宣布了旧金山巨人队输掉比赛的消息。当时,圣地亚哥教士队凭借 80 场胜利暂列西区比赛第一,旧金山巨人队只赢得了 59 场比赛,要想追上圣地亚哥教士队,至少还得再赢 21 场比赛才行。然而,根据赛程安排,巨人队只剩下 20 场比赛没打了,因而彻底与冠军无缘(摘自<http://www.matrix67.com/blog/archives/5190>)。

有趣的是,报社可能没有发现,其实在两天以前,也就是 1996 年 9 月 8 日,巨人队就已经没有夺冠的可能了。那一天,圣地亚哥教士队还只有 78 场胜利,与洛杉矶道奇队暂时并列第一。此时的巨人队仍然是 59 场胜利,但还有 22 场比赛没打。因而,表面上看起来,巨人队似乎仍有夺冠的可能。然而,根据赛程安排,圣地亚哥教士队和洛杉矶道奇队互相之间还有 7 场比赛要打,其中必有一方会获得至少 4 场胜利,从而拿到 82 胜的总分;即使巨人队剩下的 22 场比赛全胜,也只能得到 81 胜。由此可见,巨人队再怎么努力,也不能获得冠军了。

在美国职业棒球的例行赛中,每个球队都要打 162 场比赛(对手包括但不限于同一分区里的其他队伍,和同一队伍也往往会有多次交手),所胜场数最多者为该分区的冠军;如果有并列第一的情况,则用加赛决出冠军。在比赛过程中,如果我们发现,某支球队无论如何都已经不可能以第一名或者并列第一名的成绩结束比赛,那么这支球队就提前被淘汰了(虽然它还要继续打下去)。从上面的例子中可以看出,发现并且证明一个球队已经告败,有时并不是一件容易的事。

关于这个事情有一个有趣的故事,下面是一段对话:

“看到上周报纸上关于爱因斯坦的那篇文章了吗？……有记者请他算出三角比赛的数学公式。你知道，一支球队赢得了很多剩余的比赛，其他球队则赢这个赢了那个。这个比赛到底有多少种可能性？哪个球队更有优势？”

“他到底知道吗？”

“显然他知道的也不多。上周五他选择道奇队没有选巨人队。”

Team i	Wins w_i	Losses l_i	To play r_i	Against = r_{ij}			
				Atl	Phi	NY	Mon
Atlanta	83	71	8	-	1	6	1
Philly	80	79	3	1	-	0	2
New York	78	78	6	6	0	-	0
Montreal	77	82	3	1	2	0	-

上面的表是四个球队的比赛情况，现在的问题是哪些球队有机会以最多的胜利结束这个赛季？可以看到蒙特利尔队因最多只能取得 80 场胜利而被淘汰，但亚特兰大队已经取得 83 场胜利，蒙特利尔队因为 $w_i + r_i < w_j$ 而被淘汰。费城队可以赢83场，但仍然会被淘汰。。。。如果亚特兰大输掉一场比赛，那么其他球队就会赢一场。所以答案不仅取决于已经赢了多少场比赛，还取决于他们的对手是谁。

请利用最大流算法给出上面这个棒球问题的求解方法。

三、实验要求

1. 解释流网络的构造原理。
2. 解释为什么最大流能解决这个问题。
3. 给出上面四个球队的求解结果。
4. 尽可能实验优化的最大流算法。
- 5.对 <https://coursera.cs.princeton.edu/algs4/assignments/baseball/specification.php> 提供的测试数据进行测试

四、求解问题的算法原理描述

1、如何判断球队被淘汰

设赛区共有 n 支球队。对第 i 支球队，已知：

$w[i]$ ：当前胜场数； $l[i]$ ：当前负场数； $r[i]$ ：剩余比赛数；

$g[i][j]$ ：球队 i 和球队 j 之间尚未进行的比赛数。

若判断目标球队为 x ，则它的理论最高胜场为： $W_{\max} = w[x] + r[x]$

简单淘汰

若球队 i 的最大可能胜利场数小于球队 j 已经取得的胜利场数，即 $w_i + r_i < w_j$ ，则球队 i 必然没有夺冠的可能，可以直接被淘汰。

以上面四个球队的比赛情况为例，Montreal 最大可能胜利场数为 $77+3=80$ ，小于 Atlanta 已经取得的胜利场数 83，则 Montreal 胜利场数一定不超过 Atlanta，直接失去夺冠的可能。

非简单淘汰

不管剩余比赛场数中的胜利场数如何分配到现有队伍中，队伍 i 最大可能胜利场数都会小于某个队伍最终胜利场数，则队伍 i 被淘汰。反之，若存在一种胜利场数分配方案使得队伍 i 的最大可能胜利场数不被任何一个球队超越，则队伍 i 存在夺冠可能，所以可以不被淘汰。

以上面四个球队的比赛情况为例，如果 Philly 在其剩余的 3 场比赛均取得胜利，则最终 Philly 的胜场数为 83，此时 Atlanta 胜场数为 83，New York 的胜场数为 78，且这两者间还剩 6 场比赛要进行。

假如 6 场比赛中 Atlanta 一旦有一场比赛胜过 New York，Atlanta 的胜场数就达到 84，大于 Philly 的胜场数 83，Philly 的胜场数必然不可能再超越 Atlanta，故被淘汰。

假如 6 场比赛中 Atlanta 全部输于 New York，则 New York 的胜场数达到 84，大于 Philly 的胜场数 83，Philly 的胜场数必然不可能再超越 New York，故被淘汰。

上面描述，可以理解为 6 场比赛中一定会在这两个球队中产生 6 个胜场，而我们需要对这 6 个胜场在 Atlanta 和 New York 这两个球队中进行分配，如果将大于等于 1 个胜场数分配给 Atlanta，则 Atlanta 的胜场数就大于等于 84，大于 Philly 的胜场数 83，Philly 被淘汰；如果将全部胜场数分配给 New York，则 New York 的胜场数就达到 84，大于 Philly 的胜场数 83，Philly 同样被淘汰。

因此对于 Philly 来说，尽管它的胜利场数达到最大，但由于不管如何分配胜利场数到剩下的队伍中，都存在队伍的最终胜场数大于 Philly，所以 Philly 就失去了夺冠的可能，最终被淘汰。

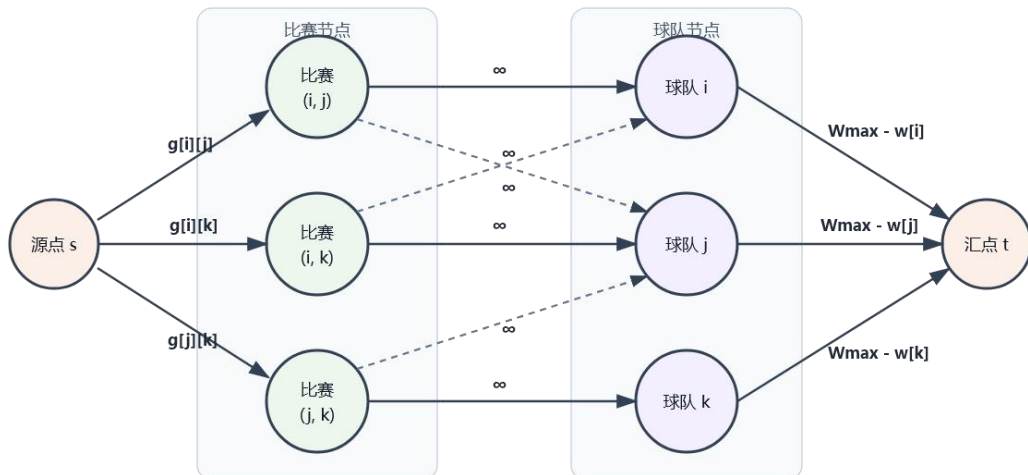
综上所述，简单淘汰很容易实现，但非简单淘汰则要考虑各球队间比赛剩余场数、输赢的可能、目前的胜利场数等等因素，来进行剩余胜利场数的分配，对于这个复杂的问题，网络流结构可以很好地适配这个剩余胜利场数分配问题。

2、流网络的构造原理

对目标球队 x ，构造一个流网络。网络只考虑不包含 x 的其他球队之间的剩余比赛，因为我们在判断 x 是否还有可能夺冠时，可以假设 x 自己剩余比赛全部获胜。

通用流网络构造模板

判断目标球队 x 是否被淘汰时，只考虑“不含 x 的剩余比赛”



若源点到所有比赛节点的边都能满流，则剩余比赛可以安排出一种不淘汰 x 的结果。

1) 流网络的 4 种节点

1. 源节点：表示起点 s 。
2. 比赛节点：每个不包含 x 的剩余比赛组合 (i, j) 对应一个节点；
3. 球队节点：除 x 外，每支球队对应一个节点；
4. 汇节点：表示终点 t 。

边和容量设置如下：

$s \rightarrow$ 比赛节点 (i, j) ，容量为 $g[i][j]$ 。

这表示球队 i 和 j 之间还有多少场比赛必须分配胜者。

比赛节点 $(i, j) \rightarrow$ 球队节点 i 和 比赛节点 $(i, j) \rightarrow$ 球队节点 j ，容量设为无穷大。

这表示这几场比赛的胜场可以分配给 i ，也可以分配给 j 。

球队节点 $i \rightarrow t$ ，容量为 $W_{\max} - w[i]$ 。

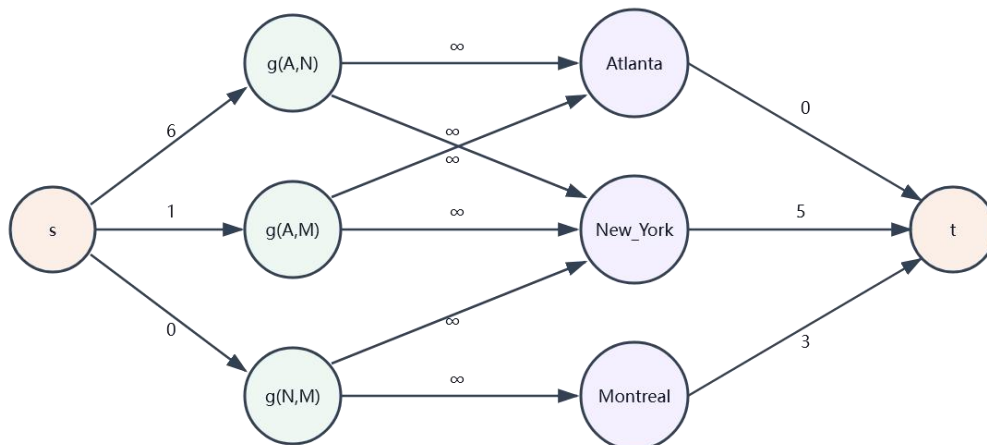
表示球队 i 最多还能赢多少场，才能保证最终胜场不超过目标球队 x 的理论最高胜场。

如果所有从源点出发的边都能被最大流完全填满，说明所有剩余比赛都可以被合理分配胜者，并且没有任何球队最终胜场超过 x 的最高胜场。因此 x 没有被淘汰。

反之，如果源点边不能全部满流，说明无论如何安排比赛结果，都会有某些球队的胜场超过 x ，因此 x 已经被淘汰。

以 Philly 为例子，希望知道 Philly 在达到最大胜利场数的情况下是否有夺冠的可能，则 $W = w_{\text{Philly}} + r_{\text{Philly}} = 80 + 3 = 83$ ，所以 Atlanta、New York、Montreal 最大剩余胜利场数分别为 0、5、6。

以 Philadelphia 为目标球队的流网络示意



源点到比赛节点容量 = 两队剩余比赛数

比赛节点到球队节点容量 = ∞ ，表示胜场可分配给任一方球队到汇点容量 = $W_{\max} - \text{当前胜场}$

3、为什么网络流结构适合该问题

1 可行赛程到可行流

假设存在一种剩余赛程结果，使目标球队 x 能够并列第一或第一。

对于任意一场不含 x 的比赛，如果胜者是球队 i ，则把 1 单位流量从对应比赛节点流向球队节点 i 。若 i 和 j 之间有多场比赛，则按照这些比赛的胜负结果分配对应数量的流

量。

由于每场比赛必然产生一个胜者，所以从源点到每个比赛节点的容量都可以被用满。又因为这种赛程保证所有球队最终胜场不超过 $W_{\max}$ ，所以球队节点到汇点的容量也不会被突破。因此，该赛程对应一个满流方案。

2 可行流到可行赛程

反过来，若最大流能使所有源点边满流，则每个比赛节点收到的流量都等于该组球队之间的剩余比赛数。

比赛节点流向两个球队节点的流量可以解释为这些比赛分别由哪支球队获胜。球队节点到汇点的容量限制保证每支球队新增胜场不超过 $W_{\max} - w[i]$ ，即最终胜场不超过目标球队 x 的理论最高胜场。因此，满流方案可以还原为一种合法的赛程结果，使 x 仍然有机会获得第一或并列第一。

3 关键定理

目标球队 x 未被淘汰，当且仅当上述流网络中所有从源点 s 指向比赛节点的边都满流。这个定理的作用是把“所有可能比赛结果的组合搜索”转化为一次最大流计算。原问题若直接枚举，每场比赛都有不同胜者选择，组合数会指数增长；最大流建模后，只需判断一个网络是否存在满足容量约束的可行流。

综合上述过程，我们能确定，将棒球淘汰问题应用到网络流结构中：

- 网络中的流 = 待分配的剩余胜利场数。
- 待分配的剩余胜利场数 = 从源节点出发边的容量和。
- 从源节点出发到汇节点的流的总量 = 在保证球队 i 不被淘汰的情况下，可分配给各球队的剩余胜利场数。
- 求该网络流的最大流，实际是在保证球队 i 不被淘汰的情况下，求最大的可分配给各球队的剩余胜利场数。
- 如果网络的最大流（最大的可分配给各球队的剩余胜利场数）= 剩余比赛场数，说明存在一种方案，使得全部剩余胜利场数分配给各球队后，能保证球队 i 不被淘汰
- 如果网络的最大流（最大的可合理分配的剩余胜利场数）< 剩余比赛场数，说明不存在任何方案，使得全部剩余胜利场数分配给各球队后，能保证球队 i 不被淘汰，即球队 i 必然被淘汰

4、求解最大流问题的算法

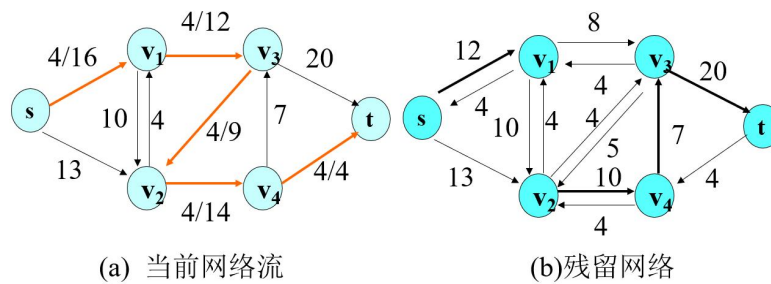
1) Ford-Fulkerson

算法原理：

Ford-Fulkerson 算法通过逐步调整流量，寻找增广路径，直到没有剩余增广路径，从而计算出最大流。

增广路径的找寻方法：利用深度优先搜索（DFS），从源节点出发，沿着当前容量大于 0 的边进行递归搜索，直到找到一个通往汇节点的路径。如果找到路径，则记录路径中的最小边容量作为可增量，并沿路径调整各边的流量；如果没有找到，则回溯并尝试其他路径。

Ford-Fulkerson 算法在找寻增广路径前，需要将网络流转化为残留网络，这是因为残留网络反映了当前流量状态下各边的剩余容量和反向容量，从而支持对流量的调整。残留网络中的每条正向边表示从一个节点到另一个节点的剩余可用容量，每条反向边表示能够减少流量的容量。



增广路径是指在流网络中从源节点到汇节点之间的路径，这条路径上的每条边都有正的剩余容量（即在这条路径上还可以增加流量）。用 DFS 在残留网络中寻找从源节点到汇节点的路径，并且确保找到的路径上每条边的剩余容量都大于零。接下来沿着找到的增广路径增加流量，根据路径上的瓶颈容量（路径上最小的剩余容量）调整流量，更新路径上的正向和反向边容量。继续在更新后的残留网络中寻找增广路径，直到找不到新的增广路径为止，表示已达到最大流量。

时间复杂度分析：

- 路径查找：使用 DFS 查找增广路径时，每次搜索的时间复杂度是 $O(E)$ ，其中 E 是网络流图中边的数量。
- 迭代次数：最多需要增加的流量与路径有关。在每次找到增广路径时，路径上的最小剩余容量（瓶颈容量）至少为 1。假设最大流的总值为 $|f|$ （流量总和），每次找到一条增广路径至少可以增加 1 单位流量。因此，最多需要 $|f|$ 次增广路径查找。
- 所以，Ford-Fulkerson 算法的时间复杂度 $O(E * |f|)$ 。
- 假设球队数为 n ，则需要进行 n 次 Ford-Fulkerson 算法，所以算法整体复杂度为 $O(n * E * |f|)$ 。

代码解释：

初始化残留网络：

```
for (u = 1; u <= graph_len; ++u)
{
    for (v = 1; v <= graph_len; ++v)
    {
        rGraph[u][v] = graph[u][v];
    }
}
```

`rGraph` 是残留网络的表示，用于追踪每条边上的剩余容量。通过遍历所有节点对 (u, v) ，将输入的流图 `graph` 的容量复制到 `rGraph` 中。

寻找增广路径并更新最大流：

```
while (1) {
    memset(visisted, 0, sizeof(visisted));

    int pathFlow = dfs(rGraph, s, t, INT_MAX, visisted);

    if (pathFlow == 0)
        return maxFlow;

    maxFlow += pathFlow;
}
```


使用 `memset` 将 `visited` 数组重置为 0，以便在每次搜索增广路径时重新标记节点。调用 `dfs` 函数在残留网络中从源点 `s` 到汇点 `t` 寻找增广路径。`dfs` 函数返回找到的增广路径的流量 `pathFlow`。如果 `pathFlow` 为 0，表示无法找到增广路径，最大流量计算结束，返回 `maxFlow`。如果找到增广路径，将 `pathFlow` 累加到 `maxFlow` 中，并继续寻找新的增广路径。

更新残留网络：

```
for (int v = 1; v <= graph_len; v++) {
    if (rGraph[u][v] > 0 && visited[v] == 0) {
        int curr = dfs(rGraph, v, sink, std::min(minEdge, rGraph[u][v]), visited);

        if (curr != 0) {
            rGraph[u][v] -= curr;
            rGraph[v][u] += curr;
            return curr;
        }
    }
}
```

在 `dfs` 回溯时，如果找到有效路径，减少边 `u->v` 的容量，增加边 `v->u` 的回退流量。

2) Edmond-Karp

算法原理：

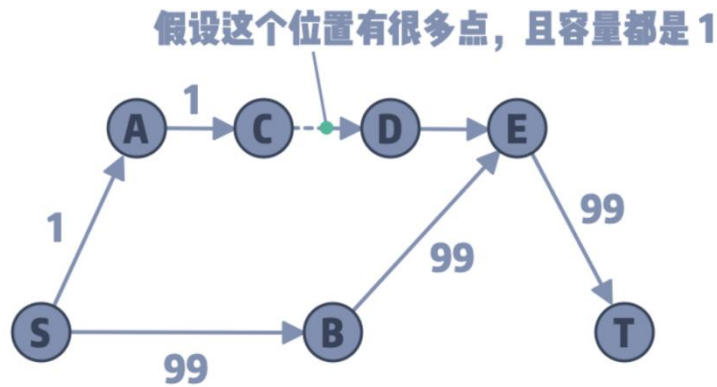
Edmond-Karp 算法通过逐步调整流量，寻找增广路径，直到没有剩余增广路径，从而计算出最大流。

增广路径的找寻方法：利用广度优先搜索（BFS）寻找增广路径的方法是从源节点开始，按层次遍历图中的边，寻找所有可能的路径中容量大于 0 的边，直到找到通往汇节点的路径。BFS 保证找到的增广路径是层次最浅的，即边数最少的，通过不断更新边的流量，层层推进，最终找到从源到汇的增广路径。

相对于 Ford-Fulkerson 算法，Edmond-Karp 算法在某些复杂网络拓扑寻找增广路径方面具有显著优势。

例如下面这张图，可以看到，只要找到 `S -> B -> E -> T` 这条增广路径，就能找到该网络的最大流 99。假设使用 Ford-Fulkerson 算法找网络最大流，则会使用 DFS 来找增广路径，那么首先会找到 `S -> A -> C -> ..... -> D -> E -> T` 这条增广路径，增广 1 的流量，因为在 DFS 搜索时出现分支，DFS 会优先选择搜索索引靠前的节点，故在 `S` 节点处往下搜索时，会往 `A` 节点处向下搜索，而错失了 `S -> B -> E -> T` 这条高容量且距离短的路径，增加了不必要的计算。而如果使用 Edmond-Karp 算法找网络最大流，则会使用 BFS 来找增广路径，首先找到的增广路径将会是距离最短的 `S -> B -> E -> T` 这条增广路径，直接增广 99 的流量。

因此，Edmonds-Karp 的 BFS 保证了每次增广路径的最短性，这在某些网络结构中，可以避免因低容量路径导致的低效流量调整。



时间复杂度分析：

- 路径查找：使用 BFS 查找增广路径时，每次搜索的时间复杂度是 $O(E + V)$ ，其中 E 是网络流图中边的数量， V 是网络流图中顶点的数量。
- 增广路径的查找总次数：由于每条最短路径的流量增加会使得至少一条边的剩余容量减少（或从正变为零），这意味着同一条路径不会被多次使用，所以在每次查找增广路径时，从源到汇的路径长度至少增加 1，最坏情况下，从源到汇的路径长度最多增加 $V - 1$ 次，即经过全部节点。总路径长度从 1 增加到 V ，每个长度最多进行 $O(E)$ 次查找。所以增广路径的查找总次数为 $O(E * V)$ 。
- 所以，Edmond-Karp 算法的时间复杂度 $O(E * V * (E + V)) = O(E^2 * V)$ 。
- 假设球队数为 n ，则需要进行 n 次 Edmond-Karp 算法，所以算法整体复杂度为 $O(n * E^2 * V)$ 。

代码解释：

记录增广路径：

```
int parent[V];
int max_flow = 0;
```

由于 BFS 找增广路径无法向 DFS 找到增广路径后通过回溯更新残留网络，所以我们需要用一个 `parent` 数组，记录增广路径上所有节点的父节点，如此将增广路径记录下来。

循环寻找增广路径并更新网络：

```
while (bfs(rGraph, s, t, parent))
{
    int path_flow = INT_MAX;
```

调用 `bfs` 函数寻找增广路径，`bfs` 返回是否存在从 `s` 到 `t` 的路径，并将路径存储在 `parent` 数组中。`path_flow` 初始化为无穷大，用于在后续步骤中找出路径上最小的剩余容量。

找到增广路径上的最小流量：

```
for (v = t; v != s; v = parent[v])
{
    // 找到增广路径上的最小流量
    u = parent[v];
    path_flow = (path_flow < rGraph[u][v]) ? path_flow : rGraph[u][v];
}
```

通过 `parent` 数组回溯从 `t` 到 `s` 的路径，找到该路径上各边的最小剩余容量 `path_flow`。该值表示该路径上可以增加的最大流量。

更新残留网络和最大流：

```
for (v = t; v != s; v = parent[v])
{
    u = parent[v];
    rGraph[u][v] -= path_flow;
    rGraph[v][u] += path_flow;
}

// 更新最大流
max_flow += path_flow;
```

根据找到的 `path_flow` 更新残留网络：从 `u` 到 `v` 的剩余容量 `rGraph[u][v]` 减少 `path_flow`。从 `v` 到 `u` 的回退流量 `rGraph[v][u]` 增加 `path_flow`。同时将 `path_flow` 加到 `max_flow`，累加到当前的最大流量中。

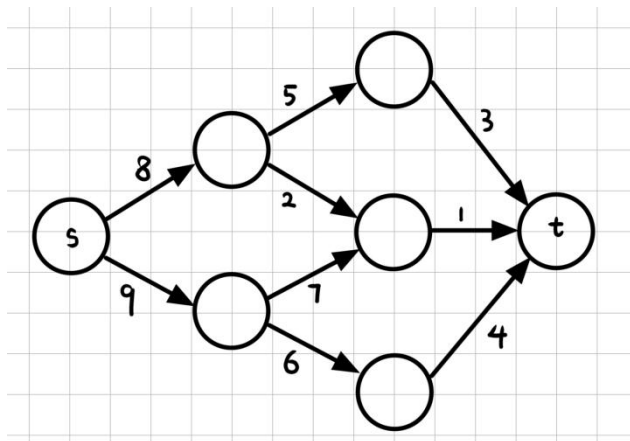
3) Dinic

算法原理：

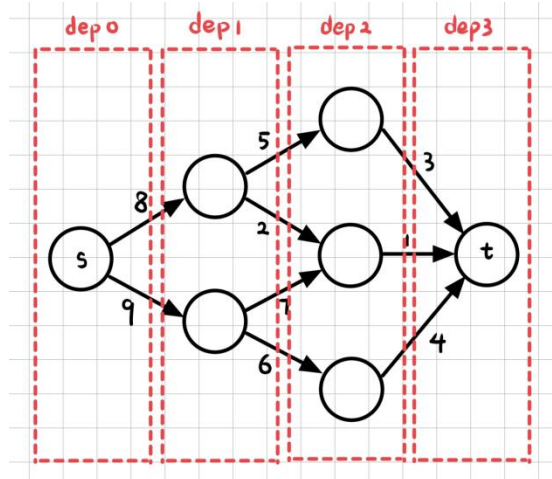
Dinic 算法通过构造层次网络，将流网络分解为多个层次，每层节点只连接到下一层，从源到汇递进搜索所有增广路径，同时在层次网络中计算最大流。

首先使用广度优先搜索（BFS）构建层次图，再用深度优先搜索（DFS）寻找所有可行的增广路径，每次搜索只能从当前层向下一层搜索，并调整路径上的流量，通过反复构建和调整，直至没有增广路径时，完成最大流计算。

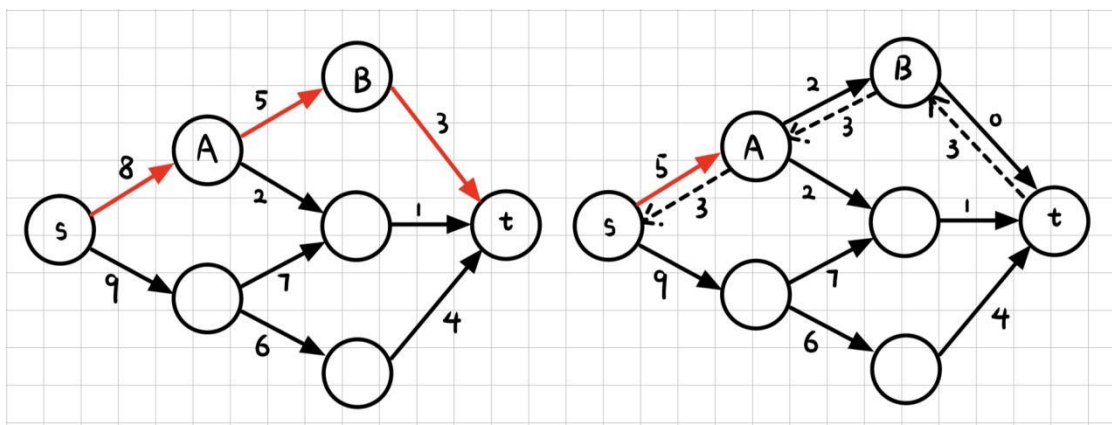
以下图为例，展示 Dinic 算法的解题过程。



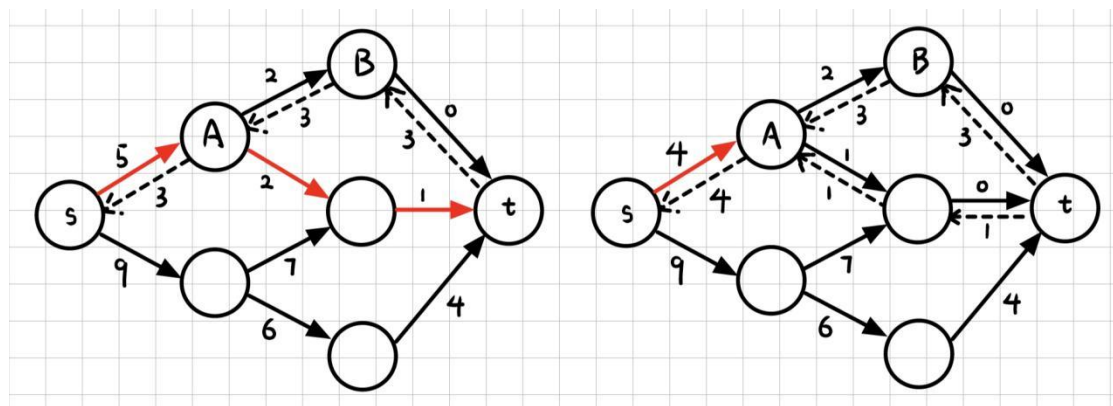
首先，使用 BFS 构建层次图，可以看到，下图被分成 4 个层次。



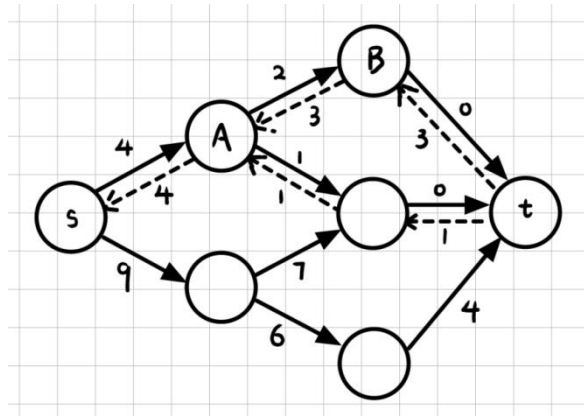
接下来，用 DFS 寻找增广路径，可以看到，路径上经过的节点的层次均不同，这便是 Dinic 算法中 DFS 找增广路径时的性质——只从当前层次向下一个层次搜索。找到源节点后回溯，更新路径上边的容量，回溯至产生分支的节点 A。由于刚刚路径上经过的最小容量为 3，所以路径上所有正向边的容量减 3，反向边的容量加 3。回溯到节点 A 并更新完路径容量后，查看到节点 A 的边容量是否大于 0，若等于 0，则继续回溯，若大于 0，则继续往下搜索。可以看到，到节点 A 的边容量为 5，大于 0，所以继续向下搜索。



从节点 A 继续向下一个分支且下一个层次搜索，找到源节点后回溯，同样的，回溯过程中更新路径上边的容量。回溯到节点 A，节点 A 的所有分支已经搜索完毕，故回溯。



从源节点出发到节点 A 的分支的增广路径搜索完毕，并且搜索过程中，所有边的容量均得到更新，接下来，会从源节点出发到下一个分支进行增广路径的搜索。



与 Edmond-Karp 比较:

由于其通过 BFS 构建层次图，搜索时也只从当前层向下一层进行搜索，所以它的搜索路径为最短路。

但它与 Edmond-Karp 算法不同之处是：最短增广路每个阶段执行完一次 BFS 增广后，要重新启动 BFS 从源点开始寻找另一条增广路；而在 Dinic 算法中，只需一次 DFS 过程就可以实现多次增广，这是 Dinic 算法的巧妙之处

时间复杂度分析:

- 构建分层网络：使用 BFS 构建分层网络时，时间复杂度是 $O(E + V)$ ，其中 E 是网络流图中边的数量， V 是网络流图中顶点的数量。
- 增广路径查找：使用 DFS 寻找增广路径，DFS 的时间复杂度为 $O(E)$ 。
- 构建分层网络次数：从源到汇的路径长度最多增加 V 次，所以次数为 $O(V)$
- 所以，Dinic 算法的时间复杂度 $O(E * V * (E + V)) = O(E^2 * V)$
- 假设球队数为 n ，则需要进行 n 次 Edmond-Karp 算法，所以算法整体复杂度为 $O(n * E^2 * V)$

代码解释:

循环构建分层网络并查找阻塞流:

```
int max_flow = 0, tf = 0;
while (bfs(rGraph, s, t)) {
    while (tf = dfs(rGraph, s, t, INT_MAX)) {
        max_flow += tf;
    }
}
return max_flow;
```

bfs 函数用于构建分层网络，判断是否存在从源点 s 到汇点 t 的增广路径。在构建分层网络后，dfs 函数查找阻塞流并计算可增加的流量 tf （阻塞流是指从源点到汇点的路径，在这条路径上，不同层次的节点只按顺序访问且沿途边的剩余容量最大限度的减少）。当 tf 不为 0 时，增加 tf 到 max_flow ，并更新残留网络中的容量。最终，当 bfs 找不到新的增广路径时，返回 max_flow 作为最终的最大流量。

五、算法测试结果及效率分析

对于四个球队的比赛情况，它们淘汰结果如下。

Team i	Wins w_i	Losses l_i	To play r_i	Against = r_{ij}			
				Atl	Phi	NY	Mon
Atlanta	83	71	8	-	1	6	1
Philly	80	79	3	1	-	0	2
New York	78	78	6	6	0	-	0
Montreal	77	82	3	1	2	0	-

球队	最高可能胜场	是否被淘汰	淘汰证书 R	说明
Atlanta	91	否	无	存在一种剩余赛程安排，使其他球队最终胜场不超过 Atlanta
Philadelphia	83	是	{ Atlanta, New_York }	非平凡淘汰，需要通过最大流和最小割得到证书
New_York	86	否	无	存在合法赛程使 New_York 仍可并列第一或第一
Montreal	80	是	{ Atlanta }	平凡淘汰，因为 Atlanta 当前已有 83 胜，超过 Montreal 的最高可能胜场

下面从普林斯顿大学课程网站获取 5 个测试数据，对上面提出的算法效率进行测试。


BASEBALL ELIMINATION

[Spec](#)
[FAQ](#)
[Project](#)
[Submit](#)

• The flow networks that arise in the baseball elimination problem have special structure, e.g., they are bipartite and the edge capacities are small integers. As a result, the Ford-Fulkerson algorithm performs significantly faster than its worst-case guarantee of V^2 .

INPUT AND TESTING

Input. The zip file [baseball.zip](#) contains some sample input files.

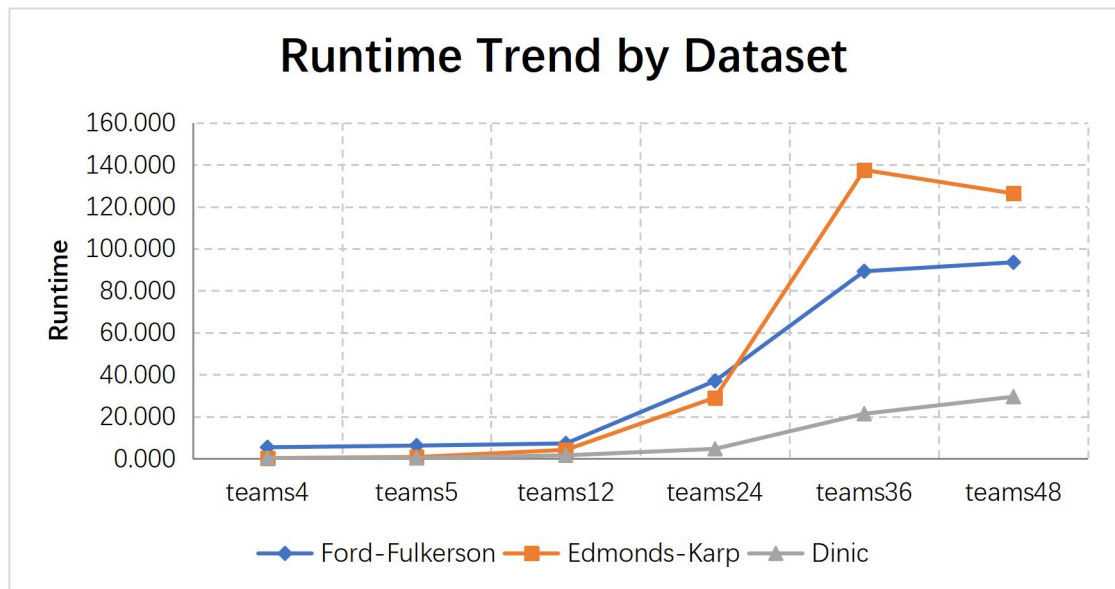
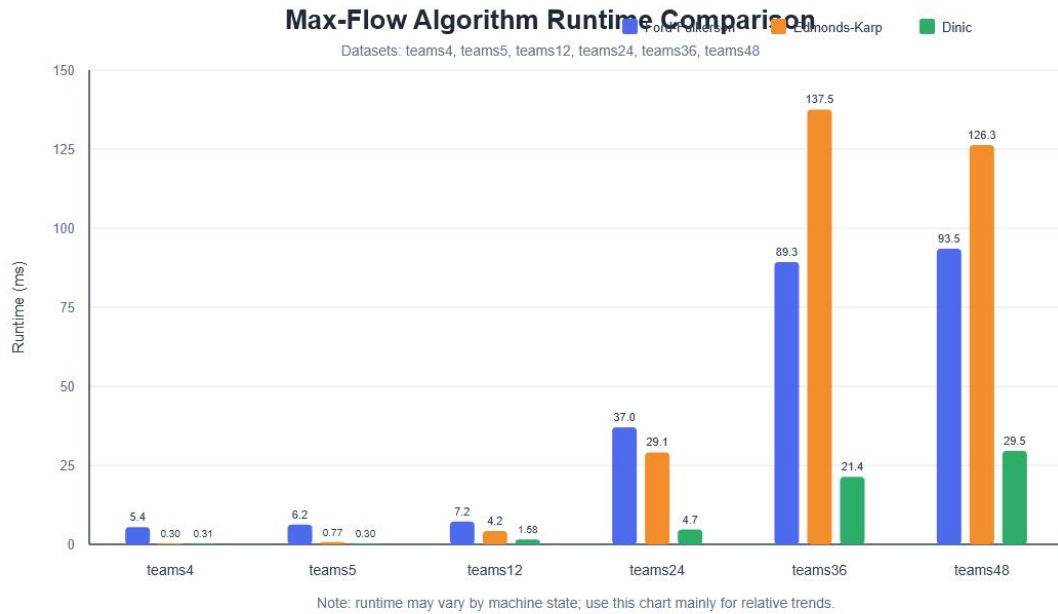
Testing. For reference, the teams below are mathematically eliminated for *nontrivial* reasons. By nontrivial, we mean that the certificate of elimination requires two or more teams. If a team is trivially eliminated, it does not appear in the list below.

- [teams4.txt](#): Philadelphia.
- [teams4a.txt](#): Ghaddafi.
- [teams5.txt](#): Detroit.
- [teams7.txt](#): Ireland.
- [teams24.txt](#): Team13.
- [teams32.txt](#): Team25, Team29.
- [teams36.txt](#): Team21.
- [teams42.txt](#): Team6, Team15, Team25.
- [teams48.txt](#): Team6, Team23, Team47.
- [teams54.txt](#): Team3, Team29, Team37, Team50.

To verify that you are returning a valid certificate of elimination R , compute $a(R) = (w(R) + g(R)) / |R|$, where $w(R)$ is the total number of wins of teams in R , $g(R)$ is the total number of remaining games between teams in R , and $|R|$ is the number of teams in R . Check that $a(R)$ is greater than the maximum number of games the eliminated team can win

各最大流算法的测试结果如下。

Dataset	Ford-Fulkerson	Edmonds-Karp	Dinic
teams4	5.427	0.304	0.307
teams5	6.215	0.767	0.301
teams12	7.242	4.214	1.580
teams24	37.035	29.076	4.677
teams36	89.268	137.526	21.378
teams48	93.533	126.324	29.517



从图中可以看出，在小规模数据上三种算法的差距不明显；当数据规模增大到 teams24.txt、teams36.txt、teams48.txt 时，Dinic 的运行时间明显低于另外两种算法。这说明 Dinic 的分层图和阻塞流机制能够减少重复寻找增广路的开销，更适合较大规模的棒球淘汰网络。

六、经验总结

本实验的关键在于识别棒球比赛淘汰问题中的两个约束：

1. 每一场剩余比赛必须产生一个胜者；
2. 若目标球队仍有夺冠可能，则其他球队的最终胜场不能超过目标球队的理论最高胜场。

最大流网络恰好能够同时表达这两个约束。源点到比赛节点的容量保证所有剩余比赛都被安排；比赛节点到球队节点的流向表示胜场分配；球队节点到汇点的容量限制其他球队最多还能赢多少场。

当最大流能够填满所有源点边时，说明存在一种合法赛程使目标球队不被淘汰；否则，目标球队已经被数学淘汰。最小割进一步给出淘汰证书，使算法不仅能判断结果，还能解释原因。

在算法实现上，本实验进一步比较了 Ford-Fulkerson、Edmonds-Karp 和 Dinic 三种最大流算法。三种算法得到的淘汰结论一致，说明问题建模和程序实现具有可靠性；从运行时间看，Dinic 在较大样例上表现更好，因此适合作为本问题的主要求解算法。

指导教师批阅意见：

成绩评定：

指导教师签字：
年 月 日

备注：

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。